Arquitectura de una aplicación generativa, paso a paso (Bloque 8) / El costo real de una app de IA, calculado en vivo

✓ **Hecho:** Ver

El costo real de una app de IA, calculado en vivo

En la lección anterior viste que cada respuesta de Gemini trae, dentro de "usageMetadata", el conteo exacto de tokens que usó esa petición: cuántos enviaste ("promptTokenCount") y cuántos generó el modelo ("candidatesTokenCount").

> Esta lección usa exactamente ese dato para responder la pregunta que en algún momento te va a hacer cualquier persona responsable de un producto: ¿cuánto cuesta, en dinero real, cada vez que tu aplicación le habla al modelo?

No necesitas instalar nada nuevo para esta lección: vas a agregar una función más al mismo módulo que ya armaste en el Bloque 4, con "**armar_peticion**", "**enviar_peticion**" y "**extraer_texto**" ya funcionando. Con tu entorno virtual activo alcanza.

🧮 De tokens a costo: la fórmula

Cada proveedor de modelo publica un precio por token, casi siempre expresado como precio por cada mil o cada millón de **tokens**.

> ⚖️ **Ese precio no es uno solo**: los proveedores casi siempre cobran distinto por los tokens que tú envías (entrada) y los que el modelo genera (salida), porque generar texto le cuesta más al proveedor que simplemente leerlo.

La fórmula, entonces, es directa: el costo de una petición es la cantidad de tokens de entrada multiplicada por su precio, más la cantidad de tokens de salida multiplicada por el suyo.

```
costo = (promptTokenCount * precio_por_token_entrada) + (candidatesTokenCount * precio_por_
```

Los valores exactos de **"precio_por_token_entrada"** y **"precio_por_token_salida"** cambian con el tiempo y dependen del modelo específico que uses, así que siempre se confirman en la página de precios vigente del proveedor, no se memorizan.

> **Lo que sí es estable, y es lo que importa aprender aquí, es la fórmula**: de dónde sale el costo y por qué depende de dos números distintos, no de uno solo.

⚙️ Llevado a código: una función más, junto a las que ya tienes

Así como **"extraer_texto"** recibe la respuesta cruda del modelo ("datos") y saca de ahí el texto generado, puedes agregar una función hermana que reciba esa misma respuesta y calcule el costo, leyendo el mismo bloque **"usageMetadata"** que ya viste:

```python
def calcular_costo(datos, precio_entrada, precio_salida):
    uso = datos["usageMetadata"]
    tokens_entrada = uso["promptTokenCount"]
    tokens_salida = uso["candidatesTokenCount"]
    return (tokens_entrada * precio_entrada) + (tokens_salida * precio_salida)
```

"calcular_costo" *recibe el mismo* ***"datos"*** *que ya le pasas a* ***"extraer_texto"****: ambas funciones leen la misma respuesta cruda, cada una buscando algo distinto dentro de ella.* ***"precio_entrada"*** *y* ***"precio_salida"*** *se le pasan como parámetros, en vez de escribirlos fijos dentro de la función, precisamente porque esos valores cambian según el modelo y el proveedor.*



Buenas prácticas

En proyectos más grandes, o cuando compartes código con otras personas, es común agregar **"type hints"** y un **docstring** a funciones como esta, por ejemplo **"def calcular_costo(datos: dict, precio_entrada: float, precio_salida: float) -> float:"**, con una primera línea que explique qué hace y qué devuelve. Ninguna función de tu módulo los usa todavía, así que no hace falta que empieces por aquí: es una práctica para tener presente más adelante, no un requisito de esta lección.



🧮 Un ejemplo completo, con números

Veamos la función anterior funcionando con datos concretos. Imagina que una petición de prueba de tu miniaplicación devuelve esta respuesta:

```
{
   "candidates": [ ... ],
   "usageMetadata": {
     "promptTokenCount": 850,
     "candidatesTokenCount": 320,
     "totalTokenCount": 1170
   }
 }
```

Y que, para este ejemplo, el proveedor cobra 0.00000010 dólares por cada token de entrada y 0.00000040 dólares por cada token de salida (números inventados, solo para ver la mecánica funcionando). Llamar a la función se ve así:

la función se ve así:

```
datos = {"usageMetadata": {"promptTokenCount": 850, "candidatesTokenCount": 320, "totalToke

costo = calcular_costo(datos, 0.00000010, 0.00000040)
print(costo)
# 0.00021299999999999997
```

Ese resultado sale de exactamente los dos términos de la fórmula: 850 tokens de entrada por 0.00000010 son 0.000085 dólares, más 320 tokens de salida por 0.00000040 son 0.000128 dólares. Sumados, 0.000213 dólares por esa única petición, menos de una centésima de centavo.

Si ejecutas exactamente este código, es probable que la consola no muestre "0.000213" limpio, sino algo como "0.00021299999999999997". No es un error tuyo ni notación científica: es la forma en que las computadoras representan fracciones decimales en binario, que casi nunca es exacta. Para ver el número limpio, redondea antes de imprimir: "round(costo, 6)" o "print(f'{costo:.6f}')" muestran "0.000213" directamente.

> **Los precios de este ejemplo son inventados a propósito**, para que el cálculo sea fácil de seguir a mano. Los precios reales de cada modelo, siempre en la página de **precios** vigente del **proveedor**, van a ser distintos, pero la mecánica (multiplicar cada conteo de tokens por su precio y sumar) es exactamente la misma.

Posibles errores comunes

¿Qué hacer si ves **"KeyError: 'usageMetadata'"**? Pasa lo mismo que ya viste con **"extraer_texto"** y el campo **"candidates"**: si la petición falló, quedó bloqueada, o la respuesta llegó incompleta, ese campo simplemente no está en **"datos"**. No hace falta un mecanismo nuevo: la misma llamada protegida con **"try/except"** que ya armaste al cerrar el Bloque 4 captura este caso exactamente igual.

> **Importante:** ¿Por qué el resultado no sale "limpio" en la consola? Eso no es un error: es la imprecisión de punto flotante que se explica justo arriba.



💰 Por qué "calculado en vivo" importa más de lo que parece

Es tentador estimar el costo de una aplicación mirando solo el largo del texto que se envía, en caracteres o en palabras. Ya viste en la lección anterior por qué eso es impreciso: el número real de tokens depende de cómo el modelo **tokeniza** ese texto específico, no de cuántos caracteres tiene.

> 💡 **El costo real se conoce a partir del consumo**
>
> Calcularlo en vivo, a partir de **"usageMetadata"**, es la única forma de tener el número exacto, no una aproximación.
>
> Esa diferencia importa especialmente con **"candidatesTokenCount"**: tú puedes controlar

más o menos cuánto texto envías, pero cuánto texto genera el modelo depende de la respuesta misma, y puede variar de una petición a otra aunque la pregunta sea parecida.

Estimar el costo antes de hacer la petición es, en el mejor de los casos, un cálculo aproximado; calcularlo después, con el dato real que trae la respuesta, es exacto.

📈 De una petición a una aplicación completa

El costo de una sola petición de prueba, como las que hiciste hasta ahora, es una fracción mínima de un centavo: prácticamente irrelevante por sí sola.

> 🌐 Pero una **aplicación real** no hace una petición, hace miles o millones, y ese costo por petición se multiplica directamente por el volumen de uso.

Esto es, en el fondo, la primera pieza concreta de una decisión de arquitectura: cuántos tokens le pides al modelo que lea y que genere en cada interacción no es solo una cuestión de calidad de la respuesta, es también una cuestión de costo, y ese costo se multiplica por cada usuario y cada interacción de tu aplicación.

Es exactamente el tipo de compensación que vas a evaluar más adelante en esta unidad, cuando compares modelos candidatos no solo por lo bien que responden, sino también por lo que cuesta cada respuesta.

Antes de pasar a la actividad de diagramación: agrega "calcular_costo" a tu módulo y pruébala con el mismo "datos" de este ejemplo, cambiando los precios o los conteos de tokens por otros números. No hace falta una petición real, alcanza con confirmar que el resultado cambia exactamente como esperarías según la fórmula.



Siguiente actividad

Diagrama la arquitectura · Actividad práctica

Con la fórmula de **costo** ya clara, y sabiendo exactamente de dónde sale cada número que la compone, estás listo para la actividad práctica de este **bloque**: diagramar la arquitectura de tu propia **miniaplicación**, incorporando tanto el ciclo de tokens como el costo que acabas de ver.

Última modificación: jueves, 24 de septiembre de 2026, 16:10